LOGICIEL PHARMACIE

Phase 5 - Prototype frontend code-first et workflow Vibe Coding V0.1

Dossier consolidé de référence pour ChatGPT, Codex et la validation humaine

Statut : PLAN VALIDÉ - EXÉCUTION À DÉMARRER

Date de consolidation : 2 août 2026

1. Décision exécutive

La méthode frontend de la V1 est officiellement orientée code-first. Figma n'est pas obligatoire. Le prototype sera construit directement avec WPF et Blazor Hybrid, exécuté avec des données synthétiques, évalué par captures et tests manuels, puis corrigé de manière itérative par Codex.

Élément	Décision
Support principal	Code exécutable WPF + Blazor Hybrid
Base visuelle	Composants Fluent UI Blazor ou équivalent validé
Maquettes Figma	Facultatives
Données	Exclusivement synthétiques pendant le prototypage
Responsable de conception	ChatGPT comme co-ingénieur et manager technique
Responsable d'implémentation	Codex, sprint par sprint
Validation	Automatique par Codex + manuelle par l'utilisateur
Gate final	Décision PASS/FAIL après revue du rapport et du rendu

2. Pourquoi la méthode code-first est retenue

- Le produit final est une application desktop exécutable, pas une maquette.
- L'utilisateur n'a pas à devenir designer ni à déplacer manuellement des éléments dans Figma.
- Les règles fonctionnelles, les parcours et les contraintes sont déjà documentés dans les Phases 1 à 4.
- Les composants Razor, le HTML et le CSS sont bien adaptés à la génération et à l'itération assistées par Codex.
- Les tests de clavier, résolution, chargement, erreur et navigation sont possibles sur l'interface réelle.
- Les tokens globaux permettent de modifier couleurs, espacements et rayons sans retoucher chaque écran.

La méthode ne consiste pas à demander à Codex de « créer tout le frontend » en un seul prompt. Le travail reste découpé en sprints autonomes, chacun avec un périmètre, des critères d'acceptation, des tests et un rapport obligatoire.

3. Architecture du prototype frontend

Couche	Responsabilité
Pharmacy.Desktop / WPF	Fenêtre native, démarrage, BlazorWebView, intégration Windows, périphériques et erreurs fatales.
Composants Blazor Hybrid	POS, catalogue, stock, achats, caisse, crédits, reporting et administration.
Design system	Tokens CSS, composants communs, états, focus clavier, densité et cohérence visuelle.
Services de démonstration	Données synthétiques et scénarios simulés, sans SQL Server réel au premier vertical slice.
Pharmacy.Application	Cas d'utilisation réels branchés après validation visuelle des écrans.

4. Design system code-first

- Palette claire, professionnelle et neutre ; couleur principale bleu-vert provisoire.
- Rouge réservé aux erreurs, orange aux avertissements et vert aux succès.
- Espacements centralisés : 4, 8, 12, 16, 24, 32 et 48 px.
- Typographie lisible ; total du POS fortement mis en avant.
- Composants communs pour boutons, champs, tableaux, dialogues, badges, alertes, chargements et états vides.

- États obligatoires : default, hover, focus, active, disabled, loading, error et success.
- Aucune information critique ne dépend uniquement de la couleur.

Un écran interne UI Lab sera disponible en développement pour visualiser tous les composants et leurs états. Il ne fera pas partie de l'interface normale de production.

5. Workflow officiel de Vibe Coding

1. ChatGPT définit le sprint, les décisions, le périmètre, les risques et les critères d'acceptation.
2. ChatGPT fournit un prompt Codex autonome, détaillé et sécurisé.
3. Codex lit les documents, inspecte le dépôt et vérifie les prérequis.
4. Codex implémente uniquement le périmètre autorisé.
5. Codex exécute restore, build, tests et validations prévues.
6. Codex corrige ses propres erreurs dans le périmètre du sprint.
7. Codex produit un rapport final structuré et ne fait aucun commit ni push.
8. L'utilisateur lance l'application et réalise la checklist manuelle.
9. L'utilisateur colle le rapport et ses observations dans la discussion.
10. ChatGPT prononce PASS, PASS WITH CORRECTIONS, FAIL ou BLOCKED.

6. Répartition des responsabilités

Acteur	Responsabilités
ChatGPT	Co-ingénierie, architecture, management, prompts, revue des rapports, décisions de Gate.
Codex	Implémentation, tests automatiques, vérification du diff, documentation et rapport.
Utilisateur	Exécution locale, contrôle visuel, test manuel et retour factuel.
Pharmacie pilote	Validation terrain des parcours réels avant production.

7. Structure obligatoire d'un prompt Codex

11. 1. Rôle et niveau d'expertise attendu
12. 2. Mission exacte du sprint
13. 3. Sources de vérité à lire
14. 4. État actuel du dépôt
15. 5. Prérequis et contrôles initiaux
16. 6. Périmètre autorisé
17. 7. Hors périmètre explicite
18. 8. Contraintes d'architecture
19. 9. Règles fonctionnelles applicables
20. 10. Exigences UX/UI ou techniques
21. 11. Sécurité, confidentialité et données
22. 12. Plan d'exécution demandé
23. 13. Tests et critères d'acceptation
24. 14. Validation manuelle à préparer
25. 15. Rapport final obligatoire et restrictions Git

8. Format obligatoire du rapport Codex

- Statut final

- Résumé du travail
- Documents consultés
- État initial observé
- Fichiers créés
- Fichiers modifiés
- Implémentation réalisée
- Décisions techniques locales
- Règles métier appliquées
- Tests ajoutés
- Commandes exécutées
- Résultats des tests
- Validation visuelle
- Problèmes rencontrés
- Écarts par rapport au prompt
- Risques et limites
- Validation manuelle requise
- État Git
- Recommandation finale

9. Découpage initial recommandé

Sprint	Résultat exécutable
DOC-0	Package documentaire Codex et règles de gouvernance.
SPRINT-0	Initialisation propre du dépôt et de la solution, sans fonctionnalité métier.
UI-0	Fondations frontend, BlazorWebView, tokens, composants communs et UI Lab.
UI-1	Shell, navigation, connexion et états système synthétiques.
UI-2	POS nominal : panier, scan simulé, paiement espèces et succès.
UI-3	POS avancé : erreurs, suspension, carte, crédit et impression échouée.
UI-4	Catalogue et fiche produit.
UI-5	Stock, lots, péremptions et inventaire.
UI-6	Achats, commandes et réceptions.
UI-7	Caisse, clients et crédits.
UI-8	Reporting, administration et diagnostic.

10. Validation manuelle du frontend

- L’application démarre sans erreur.
- Le shell s’affiche correctement en 1366 × 768 et 1920 × 1080.
- Aucun élément important n’est coupé ou masqué.
- Le clavier et le focus visible fonctionnent.
- Les boutons loading/disabled empêchent la double exécution.
- Les états vides, erreurs et accès refusé sont compréhensibles.
- Le POS conserve le total et l’action principale visibles.
- Une erreur d’impression indique clairement que la vente est enregistrée.
- Les données visibles sont synthétiques.

11. Restrictions permanentes

- Aucun commit, push ou changement de branche sans autorisation explicite.
- Aucune donnée réelle dans le code, les seeds, les tests, les captures ou les rapports.
- Aucun accès direct du frontend au DbContext.
- Aucune modification silencieuse des ADR ou des règles métier.
- Aucun ajout de microservice, cloud obligatoire ou dépendance lourde sans décision validée.
- Aucun test désactivé uniquement pour obtenir un build vert.
- Aucune migration de production exécutée automatiquement.
- Toute contradiction documentaire doit être signalée et non arbitrée par Codex.

12. Gate de validation de la Phase 5

Verdict	Condition
PASS	Résultat exécutable, tests automatiques passés, validation manuelle réussie et aucun défaut bloquant.
PASS WITH CORRECTIONS	Cœur fonctionnel correct, corrections limitées documentées avant la suite.
FAIL	Critère essentiel non satisfait, architecture ou comportement incorrect.
BLOCKED	Prérequis absent, contradiction ou décision métier manquante.

Le plan de la Phase 5 est validé. Son Gate d’exécution final restera ouvert tant que le prototype code-first n’aura pas été construit, vérifié et testé avec la pharmacie pilote.

13. Checklist de démarrage

- Package documentaire présent dans docs/.
- Tous les PDF de référence sont accessibles à Codex.
- PROJECT_CONTEXT.md, DECISIONS_REGISTER.md, SCOPE.md, CURRENT_STATE.md et CODEX_WORKFLOW.md sont présents.
- Le premier sprint ne contient aucun développement hors périmètre.
- Le prompt impose un rapport final et interdit commit/push.
- Le résultat de chaque sprint est lancé et vérifié avant le sprint suivant.